

Person 2 — ETA & Prediction Engine

Smart Rural Bus Tracking System (WISH)

1. What this part of the system does

Picture the whole system like a relay race:

Bus (ESP32 + GPS + GSM) --> Server [me] --> Database --> Dashboard / Display / SMS / IVR

The **bus** sends location updates every 2–3 minutes. **My piece** sits in the middle. It reads those updates, figures out where the bus is, when it will reach each stop, and whether anything is wrong. It writes the answer to the database. Then Person 3's SMS/IVR/Display layer reads from the database and tells the passenger.

The four things my engine does:

1. **ETA calculation** — work out how many minutes until the bus arrives at each upcoming stop.
2. **Network-loss prediction** — when the bus stops sending updates (bad signal in a rural area), make an educated guess about where it is.
3. **Historical learning** — get better at predictions over time by remembering how long past trips actually took.
4. **Route adherence** — check that the bus is on its planned route; if it isn't, raise an alert.

Three deliverables for the presentation: ETA workflow diagram, prediction workflow diagram, geofence equation.

What the bus actually sends

The GPS chip itself only knows four things from satellites: latitude, longitude, speed (in **knots** — a sailing unit!), and UTC time. The little computer on the bus (the microcontroller) does some bookkeeping before sending it to my server:

1. **Convert knots to a sensible unit.** 1 knot = 1.852 km/h, or about 0.514 m/s. My engine works in metres and seconds internally, so I'd prefer the bus to send m/s on the wire.
2. **Tag it with which bus and which route.** Each bus has a fixed bus_id and route_id baked into its firmware (e.g. BUS_01, ROUTE_05).
3. **Optionally include battery %** so the dashboard can warn before a unit dies.
4. **Save updates locally when offline.** If there's no signal, the bus stores updates on an SD card and uploads them all when signal returns.

So one packet looks like:

```
{
  "bus_id":    "BUS_01",
  "route_id":  "ROUTE_05",
  "latitude":  31.7754,
  "longitude": 76.9861,
  "speed":     35.2,
  "timestamp": "2026-06-05T16:10:00Z",
  "battery":   "82%"
}
```

What my engine writes out

Field	What it means in plain English
position	Where the bus is — either where it really is (live), or where I think it is (estimated).
next_stop	Which stop it's heading to next.
eta[]	Minutes until it reaches each upcoming stop.
status	One of: LIVE (I just heard from it), ESTIMATED (guessing), LAST_KNOWN (giving up), OFF_ROUTE (something is wrong).
confidence	A number from 0 to 1 — how sure I am. Starts at 1 when a fresh update arrives, drops the longer I go without one.

2. Calculating ETA

The basic idea: **ETA = how far the bus has to travel ÷ how fast it's likely to go**, plus the time it spends waiting at each stop along the way. The interesting part is *how* we figure out “how fast it's likely to go” — that's where the system gets smarter over time.

2.1 Tier A — Day 1, no history yet

When we first switch the system on, we have no past trips to learn from. So we just pick a sensible default speed for the route — say **30 km/h** for a rural state highway. Then for any upcoming stop:

$$\text{ETA} = \text{remaining distance to that stop} \div 30 \text{ km/h}$$

That's it. Crude, but it works on day one and is the standard way pilot projects (like the Blacksburg, Virginia rural transit pilot) start.

2.2 Tier B — After a few days of data: learn each segment of the road

Real roads aren't uniform. The stretch from stop 1 to stop 2 might be a smooth highway (avg 50 km/h), while stop 3 to stop 4 goes through a busy village (avg 20 km/h). Lumping them together at “30 km/h” makes the ETA worse, not better.

So we split the route into segments (one segment between every pair of consecutive stops) and learn an average travel time for *each segment*. We also break the day into **half-hour time buckets** (and weekday vs. weekend) because a segment behaves differently at 8 AM rush hour vs. midnight.

Now the ETA for any upcoming stop is:

ETA = how much of the current segment is left + sum of average times for every later segment + sum of typical wait time at each stop along the way

The formal version (so the team has the actual formula on paper):

$$\text{ETA}(P_m) = \underbrace{\frac{L_k - s_k}{L_k} \cdot \tau_k(b)}_{\text{remaining current segment}} + \underbrace{\sum_{i=k+1}^{m-1} \tau_i(b)}_{\text{later segments}} + \underbrace{\sum_{j=k+1}^{m-1} d_j(b)}_{\text{waits at stops}}$$

Reading the symbols: L_k is the length of the current segment, s_k is how far into it the bus already is, $\tau_i(b)$ is the average travel time we've learned for segment i during this time-of-day bucket b , and $d_j(b)$ is the average wait time at stop j .

It's the same plain-English formula written in math: **(time left in this segment) + (time for the rest of the segments) + (time waiting at each stop)**.

2.3 Tier C — Live correction (when traffic isn't normal today)

Sometimes the historical average lies — there's a cow on the road, a school just let out, it's market day. The bus is moving slower *right now* than it usually does on this segment.

To handle this we mix two estimates for the *current* segment only:

- The historical average we learned (Tier B).
- The live speed observed in the last few packets.

We tilt towards the live speed (it knows about today's problem), but for *later* segments we keep trusting history (today's cow won't still be there five segments later).

2.4 How the learning actually happens (one number per segment)

When the bus crosses a stop, we know exactly how long that segment took. We update the learned average using a trick called **exponential smoothing** — basically, the new average is mostly the old average with a little nudge from the new measurement:

new average = (0.25 × today's observation) + (0.75 × old average)

Why this trick? Because it costs us **one float per (segment, time-bucket)** — no machine learning library, no big training data — and research shows it rivals fancier methods when data is sparse. Perfect for a rural server.

The same idea is used to learn how long the bus typically waits at each stop.

2.5 Edge cases we have to handle

Situation	What we do
Bus reports speed = 0 but we can see it moved	Compute speed ourselves from two consecutive positions: distance ÷ time.
First packet of a trip, bus is suspiciously far from any segment	Mark as OFF_ROUTE rather than guessing badly.
Bus skips a stop (express service)	Detect from the geofence crossing without a speed drop; don't pollute that stop's wait-time average.
Bus clock disagrees with server clock	Trust the GPS timestamp. Fall back to server-receive time only if the GPS fix is junk (high HDOP).
Trip ends mid-route (driver change)	If "time to end of route" exceeds 2× the schedule, force-close the trip — don't let garbage into the averages.

3. What to do when the bus stops sending updates

This happens *constantly* in rural areas. We need to keep showing the passenger something useful.

3.1 Three states based on how long it's been silent

Let Δt = "time since the bus's last packet". Decisions:

- **Under ~4 minutes:** still LIVE. Updates come every 2–3 min, so 4 min of silence is normal jitter. Don't do anything.
- **4 to 15 minutes:** ESTIMATED. The bus is probably still moving — guess where.
- **More than 15 minutes:** LAST_KNOWN. After 15 min our guess is worse than just admitting "we don't know" — so we freeze the last seen position and stop pretending to have an ETA.

3.2 How we guess where the bus is (dead reckoning)

The textbook approach is called *dead reckoning*: **last known position + (speed × time elapsed) in the direction the bus was heading**. Sailors and pilots have used it for centuries when they lose their fix.

Problem: pure dead reckoning **drifts sideways** — heading is noisy, so after a few minutes the guess wanders off the road into someone's field.

The trick we use: we already know the bus's route is a fixed polyline. So instead of letting the guess float in 2D space, we *snap it onto the route* and just slide it forward along the road. Sideways error disappears entirely; only "how far along" error remains, which is bounded by how well we know the typical speed.

For the distance to slide forward, we blend two speed estimates:

- The bus's last reported speed (might be stale, might be wrong).
- The historical typical speed for this segment at this time of day.

The longer the blackout, the less we trust the stale reading and the more we trust history.
The math version of “trust history more as time passes” is:

$$\hat{d} = \gamma \cdot v_{\text{last}} \cdot \Delta t + (1 - \gamma) \cdot v_{\text{historical}} \cdot \Delta t \quad \text{where } \gamma = e^{-\Delta t / \kappa}$$

That $e^{-\Delta t / \kappa}$ thing is just a weight that starts at 1 (fully trust the last reading) and fades to 0 (fully trust history) over the course of ~5 minutes. Nothing scary — same idea as a phone battery icon dimming the lower it gets.

3.3 We also report how sure we are

Every estimate carries a **confidence** number from 0 to 1. Fresh packet -> confidence = 1. The longer the blackout, the lower it drops. The display can show “ETA ~12 min (estimated)” instead of pretending to be precise.

3.4 When the bus finally comes back online

We compare where we *guessed* it was vs. where it *actually* is, and log the error. That log lets us tune the system weekly: if guesses are consistently 200 m short, we know our typical-speed estimate is too low. This is also the **prediction accuracy metric** Priyanka asked for.

3.5 Why we're not using a Kalman filter on day one

Kalman filters are the textbook way to do this. They're also a tuning headache and overkill when our packets are 2–3 minutes apart (Kalman filters usually run on data 100x faster than that). The constrained dead-reckoning approach above gets ~80% of the benefit with ~5% of the code. Kalman is on the “if we have time” list, not the day-one list.

4. Geofencing — keeping the bus on its planned route

A **geofence** is just an invisible boundary drawn around something. We use it two ways:

- **Route corridor:** a band ~50 m wide on either side of the planned route. Inside the band = on route. Outside = something's wrong.
- **Stop circles:** a 30 m circle around each bus stop. Bus inside the circle + barely moving = “arrived at stop”.

To check these we need to measure distances between GPS points. That's where the geofence equations come in. (The deliverable says “geofence equation” — here are all three actually used.)

4.1 Distance between two GPS points — the Haversine formula

GPS gives us latitude and longitude, which are angles on a sphere, not flat x/y. We can't just use Pythagoras — we'd be off by metres at short range and kilometres at long range.

The **Haversine formula** is the standard way to compute the real distance between two lat/lon points on Earth's surface:

$$a = \sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos \varphi_1 \cos \varphi_2 \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)$$

$$d = 2R \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1 - a}\right)$$

In words: it takes the two points' latitudes and longitudes, accounts for the Earth being a sphere, and gives you the great-circle distance in metres. R is the Earth's radius (6 371 000 m). **You just plug numbers in; you don't need to derive it.**

It's accurate to under 3 metres for any distance under 10 km — way more accurate than GPS itself, so plenty good for us.

4.2 Perpendicular distance from bus to route — cross-track distance

The route is a sequence of points connected by straight lines. "Is the bus on route?" really means **"how far is the bus from the nearest line in the route?"**

That distance is called the **cross-track distance**. The full spherical formula is:

$$\delta_{xt} = \text{asin}\left(\sin(d/R) \cdot \sin(\theta_B - \theta_{\text{seg}})\right) \cdot R$$

where d is the distance from the start of the segment to the bus, θ_B is the bearing (compass direction) to the bus, and θ_{seg} is the bearing along the segment. The difference between the two bearings, combined with the distance, gives the sideways offset.

You almost never use this exact formula in code because...

4.3 The fast trick we actually use

Our route segments are short (< 5 km) and rural India is far from the poles. In this regime we can pretend the Earth is flat over each segment — convert lat/lon to flat x/y, then use high-school geometry to find the perpendicular distance from the bus to the line segment.

$$x = R(\lambda - \lambda_0) \cos \varphi_0, \quad y = R(\varphi - \varphi_0)$$

Then for a segment from P_i to P_{i+1} , the closest point on the segment to bus B is found by projecting:

$$t^* = \text{clip}\left(\frac{(B - P_i) \cdot (P_{i+1} - P_i)}{\|P_{i+1} - P_i\|^2}, 0, 1\right)$$

$$\text{closest point} = P_i + t^*(P_{i+1} - P_i), \quad \delta = \|B - \text{closest point}\|$$

In plain English: **find the foot of the perpendicular from the bus to the road line; if that foot falls between the endpoints, use it; otherwise use the nearer endpoint.**

This is sub-metre accurate at our scale (well below GPS noise) and roughly 10x faster than the spherical formula. We run it on every packet against every segment of the route, so speed matters.

4.4 The actual rule for “on route”

The bus is on route if and only if its distance to the nearest segment is at most W , where W is the corridor half-width — about 50 m on highways, 25 m in tighter town stretches.

In math: $\text{on_route}(B) \iff \min_i \delta_i(B) \leq W$.

4.5 The “arrived at a stop” rule

Bus is within 30 m of the stop AND moving slower than 2 m/s.

The speed check matters because we don’t want to fire “arrived” when the bus is just passing through near a stop without actually stopping.

4.6 The “departed from a stop” rule

Bus is more than 50 m from the stop AND moving faster than 3 m/s.

The extra 20 m gap (vs. the 30 m arrival radius) is **hysteresis**: it prevents the system from flickering between “arrived” and “departed” when the bus idles right at the edge.

4.7 Off-route alert — don’t be twitchy

Raise OFF_ROUTE only after 3 consecutive packets show the bus outside the corridor.

A single bad GPS reading (multipath, signal bounce off a building, momentary glitch) isn’t a divert — it’s noise. Waiting for 3 in a row means the alert latency is ~7.5 minutes (since packets are ~150 s apart), which is fine for safety. Better than crying wolf every time a satellite blinks.

5. How the learning gets stored

Everything the engine learns lives in a few small tables. (The database itself is Person 1 / Person 3’s territory; this is just what I need from it.)

Table	What's in it	Why
route_segments	The road geometry: each route's segments and stops	Static; seeded once when a route is added
segment_stats	Learned average travel time per segment per time-bucket	The “smart” part — gets better with every trip
stop_dwell	Learned average wait time per stop per time-bucket	Same idea, for stops
trip_log	Raw observations of completed segments	Audit / retraining if averages get polluted
prediction_log	When we guessed during a blackout, how wrong were we	The “accuracy metric” Priyanka asked for

A “time bucket” is **(weekday or weekend, half-hour of day)** — so $2 \times 48 = 96$ buckets. A 50-segment route stores $50 \times 96 = 4\,800$ average travel times. Tiny — SQLite handles it without breaking a sweat.

While the buckets are still filling up (first ~2 weeks), we fall back gracefully: use this bucket's average if we have ≥ 10 samples, else use the segment's overall average, else the route's overall average, else the 30 km/h default. No cliff-edge cold starts.

6. The two workflows (referenced from the deliverable diagrams)

6.1 ETA workflow — what happens on every packet

1. Packet arrives.
2. Sanity check (no impossible jumps, GPS fix isn't junk).
3. Snap the bus's position to the nearest point on its route.
4. If that snap distance is bigger than the corridor -> maybe OFF_ROUTE.
5. If the bus just crossed a stop -> record how long that segment took, update the learned average.
6. Recompute the ETA for every upcoming stop using (time left in current segment) + (history for later segments) + (typical wait at each stop).
7. Write everything to the database for Person 3's display/SMS/IVR to read.

6.2 Prediction workflow — every 30 seconds, for every bus

1. How long since this bus's last packet?
2. Under 4 min -> still LIVE, do nothing.
3. 4–15 min -> run dead reckoning: slide the bus forward along the route at the blended speed; mark as ESTIMATED with falling confidence.
4. Over 15 min -> freeze the position; status = LAST_KNOWN; show “we don't know” instead of a false ETA.
5. When a real packet finally arrives -> log how wrong we were, switch back to LIVE.

(See diagram files 1, 2, 3, 4 for the full graphical versions.)

7. What goes wrong, and how we handle it

What goes wrong	How we notice	What we do
Driver unplugs the device	No packets for ages	After 15 min -> LAST_KNOWN + dashboard alert. If the trip should have been running, flag as TAMPER.
GPS jumps several km (signal bounce off a building)	A jump implies a speed bigger than physically possible	Reject that packet, keep the previous fix.
Route data in the database is wrong	Buses keep going OFF_ROUTE at the same spot	If >50% of recent trips fail at the same lat/lon, flag the <i>route data</i> , not the buses.
Time-of-day bucket is sparse (midnight bus)	Few samples -> noisy average	Use the fallback chain (segment-overall -> route-overall -> default).
Bus clock disagrees with server clock	Negative Δt or absurdly large Δt	Clamp to a sensible range; log a clock-skew event.
Buffered data flushes out of order after a network outage	Old packets arrive after newer ones	Insert by (bus_id, gps_timestamp) so duplicates are ignored. Reprocess offline, not in the live ETA path.

8. Things I still need from the team

1. **Route data** — who's giving me the actual polylines (latitude/longitude of each road segment + each stop)? Likely Person 1 or Person 3 seeds it into the DB. Format: GeoJSON.
 2. **Packet rate** — confirm 2–3 min (Priyanka's note). Determines my thresholds.
 3. **Which clock do we trust** — GPS or server-receive? (Recommend GPS, fall back to server when GPS fix is junk.)
 4. **Corridor width W** — needs a sanity check by walking a few real rural routes; 50 m is a guess.
 5. **Dashboard contract** — what fields does Person 3's dashboard expect, and how often does it want updates?
-

9. The plan: build order

1. Stub out a Python module `eta_engine/` with files for geofencing, route handling, dead reckoning, ETA calculation, learning, and the entry point.

2. Wire it to Person 1/3's database.
3. Replay Priyanka's sample packet against a fake 5-stop route and watch all four statuses (LIVE, ESTIMATED, LAST_KNOWN, OFF_ROUTE) fire correctly.
4. Hand off the four diagrams as the deliverables.
5. For the demo, ship Tier B (segment + time-bucket learning). Live blending (Tier C) and Kalman are stretch goals.

10. The divert-alert pipeline (how my engine talks to Person 3)

When my engine raises **OFF_ROUTE** (the "3 consecutive bad packets" rule), three things have to happen at once:

1. Log it in the **Alerts** table (Person 1's database) so it's audited.
2. Tell **Person 3's notification dispatcher** to message every passenger who recently queried this bus.
3. Pop a banner on the **Authority Dashboard** so the transport department can act (replacement bus, contact driver).

10.1 Who gets messaged — the query log

Person 3's IVR layer already records every passenger who calls and asks about a bus (phone number, bus number, language auto-detected from the area code). I just SELECT from that table:

Column	What it is
phone	Passenger's number, like +91. . .
bus_id	Which bus they asked about
channel	Whether they called (IVR) or texted (SMS)
language	Auto-detected (e.g. Karnataka number -> Kannada)
ts	When they asked
notified	Have we already alerted them this incident?

Recency window: only notify people who asked in the **last 90 minutes**. Older queriers have probably already boarded a bus or moved on; messaging them is just noise.

10.2 What my engine sends Person 3

A single POST to their /notify endpoint — basically a structured "please alert these people":

```
{
  "event":      "ROUTE_DIVERT",
  "bus_id":     "BUS_17",
  "route_id":   "ROUTE_05",
  "at":         "2026-06-07T14:23:11Z",
  "last_known": { "lat": 11.234, "lon": 78.567 },
  "off_by_m":   142,
```

```

"template": "DIVERT",
"audience": "queriers_last_90min",
"severity": "HIGH"
}

```

I send a **template key** (DIVERT), not the actual message text. Person 3's layer fills in the words in each user's language. Cleaner separation of concerns.

10.3 The message templates

Key	English wording (Person 3 translates to others)
DIVERT	"Bus {bus_id} has diverted from its route. Please wait for another bus or move to {nearest_alt_stop}."
DIVERT_RESOLVED	"Bus {bus_id} is back on route. ETA at {stop}: {eta} min."
BUS_CANCELLED	"Bus {bus_id} is no longer in service. Next bus on this route: {next_bus_id} in ~{eta} min."

10.4 The authority dashboard

Same alert, different audience. The transport authority sees a live banner pop up so they can dispatch a replacement bus or call the driver. When they acknowledge -> status flips to ACKNOWLEDGED. When the bus comes back on route -> status flips to RESOLVED and we fire a follow-up "back on route" message to the same passengers.

10.5 Fits into Person 1's existing tables

I don't need any new tables on Person 1's side. Their Alerts table already has (Alert_ID, Bus_ID, Timestamp, Alert_Type, Status) — I just write a row with Alert_Type = 'ROUTE_DIVERT'. The only new table is the queries log in Section 10.1, which is logically Person 3's anyway.

10.6 The whole sequence, end to end

```

t = 0      My engine: 3rd packet from BUS_17 is 142 m off the corridor.
t = 0      status := OFF_ROUTE; confidence := 0.6
t = 0      INSERT Alerts(Bus_ID=17, Alert_Type=ROUTE_DIVERT, Status=OPEN)
t = 0      POST /notify -> Person 3 dispatcher
t = 0      POST /authority/banner -> live banner pops on authority screen
t = 0+ε    Person 3:
            SELECT phone, language, channel FROM queries
            WHERE bus_id='BUS_17'
            AND ts >= now - 90 min AND notified=false
            For each -> render DIVERT in their language -> SMS or IVR call
            UPDATE queries SET notified = true
t = 0+1m   Authority dispatcher clicks ACK

```

```

UPDATE Alerts SET Status='ACKNOWLEDGED'
...
t = T_back Bus is back inside corridor for 3 packets in a row
    status := LIVE
    UPDATE Alerts SET Status='RESOLVED'
    POST /notify {event: DIVERT_RESOLVED, ...}
    -> queriers get the follow-up "back on route" message

```

10.7 Anti-spam guards

- **One alert per (bus, trip).** If the bus re-enters and leaves the corridor again in the same trip, we don't re-page everybody.
 - **notified=true flag.** No double messages.
 - **Hourly rate limit per phone.** A passenger who asked about 5 buses doesn't get blasted with 5 alerts during a multi-incident event.
 - **Suppress marginal flickers.** If the bus is just barely off the corridor (less than $2 \times W$), assume GPS noise and don't alert.
-

11. Sources

- [Predicting Transit Vehicle Arrival Times — Glump.net](#) — classic write-up of GPS + historical-average ETA.
- [Survey of ETA prediction methods in public transport networks — arXiv 1904.05037](#) — taxonomy of methods (historical, regression, Kalman, ML).
- [Bus Travel Time Predictions Using Additive Models — arXiv 1411.7973](#) — lightweight statistical approach.
- [Dynamic Bus Travel Time Prediction Models — Hindawi](#) — Kalman + SVM hybrid (for the day we want to upgrade).
- [Dead reckoning — Wikipedia](#) — algorithm + error discussion.
- [Automotive Dead-Reckoning — IEEE](#) — formal speed+heading extrapolation between GPS fixes.
- [u-blox Automotive Dead Reckoning](#) — applied notes for fleets.
- [Movable-Type: distance, bearing, cross-track formulae](#) — canonical haversine + cross-track equations.
- [The Haversine Formula for Geospatial Distances — Product Teacher](#) — accuracy notes for geofencing.
- [Haversine formula — iGISMap](#) — worked example.
- [Optimized Bus Route Finder using Haversine — IEEE 10725509](#) — same formula applied to bus routes.
- [Forecast of Bus Passenger Traffic Based on Exponential Smoothing — ResearchGate](#) — background on the smoothing trick we use.
- [Bus Arrival Time Prediction: A Spatial Kalman Filter Approach — ResearchGate](#) — for when we eventually upgrade.